FINAL OS LAB NOTES

8TH-TASK

FIRST FIT

```
#include <stdio.h>

int main() {
    int b[10], p[10], nb, np;

    printf("Enter number of blocks: ");
    scanf("%d",&nb);

    printf("Enter block sizes:\n");
    for(int i=0;i<nb;i++)
        scanf("%d",&b[i]);

    printf("Enter number of processes: ");
    scanf("%d",&np);

    printf("Enter process sizes:\n");
    for(int i=0;i<np;i++)
        scanf("%d",&p[i]);

    for(int i=0;i<np;i++) {
        for(int j=0;j<nb;j++) {
            if(b[j] >= p[i]) {
                printf("Process %d allocated to Block %d\n", i, j);
                b[j] -= p[i];
                break;
            }
        }
    }

    return 0;
}
```

BEST FIT

```
#include <stdio.h>

int main() {
    int b[10], p[10], nb, np;

    printf("Enter number of blocks: ");
    scanf("%d",&nb);

    printf("Enter block sizes:\n");
    for(int i=0;i<nb;i++)
        scanf("%d",&b[i]);
```

```
    printf("Enter number of processes: ");
    scanf("%d",&np);

    printf("Enter process sizes:\n");
    for(int i=0;i<np;i++)
        scanf("%d",&p[i]);

    for(int i=0;i<np;i++) {
        int best = -1;

        for(int j=0;j<nb;j++) {
            if(b[j] >= p[i]) {
                if(best == -1 || b[j] < b[best])
                    best = j;
            }
        }

        if(best != -1) {
            printf("Process %d allocated to Block %d\n", i, best);
            b[best] -= p[i];
        }
    }

    return 0;
}
```

FOR WORST FIT USE GREATER THAN SYMBOL

10TH TASK

Page technique

```
#include <stdio.h>

int main() {
    int pages, frames, page_size;

    printf("Enter number of pages: ");
    scanf("%d", &pages);

    printf("Enter number of frames: ");
    scanf("%d", &frames);

    printf("Enter page size: ");
    scanf("%d", &page_size);

    int page_table[pages];

    // Input page table
```

```
    printf("Enter frame number for each page:\n");
    for(int i = 0; i < pages; i++) {
        printf("Page %d -> Frame: ", i);
        scanf("%d", &page_table[i]);
    }

    int logical_address;
    printf("Enter logical address: ");
    scanf("%d", &logical_address);

    // Calculate page number and offset
    int page_no = logical_address / page_size;
    int offset = logical_address % page_size;

    if(page_no >= pages) {
        printf("Invalid logical address\n");
        return 0;
    }

    int frame_no = page_table[page_no];

    int physical_address = frame_no * page_size + offset;

    printf("\nPage Number: %d\n", page_no);
    printf("Offset: %d\n", offset);
    printf("Frame Number: %d\n", frame_no);
    printf("Physical Address: %d\n", physical_address);

    return 0;
}
```

TASK-7
Producer consumer problem using semaphore

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define SIZE 5

int buffer[SIZE];
int in = 0, out = 0;

sem_t empty, full;
pthread_mutex_t mutex;

void* producer(void* arg) {
```

```
    int item = 1;

    while(item <= 10) {
        sem_wait(&empty);          // wait if buffer full
        pthread_mutex_lock(&mutex); // enter critical section

        buffer[in] = item;
        printf("Produced: %d\n", item);
        in = (in + 1) % SIZE;

        pthread_mutex_unlock(&mutex); // exit critical section
        sem_post(&full);             // signal full

        item++;
        sleep(1);
    }
    return NULL;
}

void* consumer(void* arg) {
    int item;

    while(1) {
        sem_wait(&full);           // wait if buffer empty
        pthread_mutex_lock(&mutex);

        item = buffer[out];
        printf("Consumed: %d\n", item);
        out = (out + 1) % SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&empty);          // signal empty

        sleep(2);
    }
    return NULL;
}

int main() {
    pthread_t p, c;

    sem_init(&empty, 0, SIZE); // initially all empty
    sem_init(&full, 0, 0);     // initially no items
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);
```

```
    pthread_join(p, NULL);
    pthread_join(c, NULL);

    return 0;
}
```

TASK-6
Concurrent execution of threads using pthread library

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

void* task(void* arg) {
    int id = ×(int×)arg;

    for(int i = 0; i < 5; i++) {
        printf("Thread %d is running (iteration %d)\n", id, i);
        sleep(1); // simulate work
    }

    return NULL;
}

int main() {
    pthread_t t1, t2, t3;
    int id1 = 1, id2 = 2, id3 = 3;

    // Create threads
    pthread_create(&t1, NULL, task, &id1);
    pthread_create(&t2, NULL, task, &id2);
    pthread_create(&t3, NULL, task, &id3);

    // Wait for threads to finish
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_join(t3, NULL);

    printf("All threads finished execution\n");
    return 0;
}
```

TASK-11
BANKERS ALGORITHM

```
#include <stdio.h>
```

```
int main() {
    int n, m;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resources: ");
    scanf("%d", &m);

    int alloc[n][m], max[n][m], need[n][m];
    int avail[m], flag[n], safe[n];

    // Input Allocation
    printf("Enter Allocation Matrix:\n");
    for(int i=0;i<n;i++)
        for(int j=0;j<m;j++)
            scanf("%d",&alloc[i][j]);

    // Input Max
    printf("Enter Max Matrix:\n");
    for(int i=0;i<n;i++)
        for(int j=0;j<m;j++)
            scanf("%d",&max[i][j]);

    // Input Available
    printf("Enter Available Resources:\n");
    for(int j=0;j<m;j++)
        scanf("%d",&avail[j]);

    // Step 1: Initialize flag and calculate need
    for(int i=0;i<n;i++) {
        flag[i] = 0;
        for(int j=0;j<m;j++) {
            need[i][j] = max[i][j] - alloc[i][j];
        }
    }

    int count = 0;

    // Step 2 & 3
    while(count < n) {
        int found = 0;

        for(int i=0;i<n;i++) {
            if(flag[i] == 0) {
                int j;
```

```
            for(j=0;j<m;j++) {
                if(need[i][j] > avail[j])
                    break;
            }

            if(j == m) {
                // Step 3
                flag[i] = 1;

                for(int k=0;k<m;k++) {
                    avail[k] += alloc[i][k];
                }

                safe[count++] = i;
                found = 1;
            }
        }
    }

    if(found == 0)
        break;
}

// Step 4
if(count == n) {
    printf("System is in SAFE state\n");
    printf("Safe sequence: ");
    for(int i=0;i<n;i++)
        printf("P%d ", safe[i]);
} else {
    printf("System is NOT in safe state (Deadlock possible)\n");
}

return 0;
}
```

TASK-4

Round Robin

```
#include <stdio.h>

int main() {
    int n, bt[10], rem[10], wt[10], tat[10], q, time = 0, done = 0;

    printf("Processes: "); scanf("%d", &n);
    printf("Quantum: "); scanf("%d", &q);
```

```
    for(int i = 0; i < n; i++) {
        printf("P%d BT: ", i+1);
        scanf("%d", &bt[i]);
        rem[i] = bt[i];
    }

    while(done < n) {
        for(int i = 0; i < n; i++) {
            if(rem[i] > 0) {
                if(rem[i] > q) {
                    time += q;
                    rem[i] -= q;
                } else {
                    time += rem[i];
                    wt[i] = time - bt[i];
                    tat[i] = time;
                    rem[i] = 0;
                    done++;
                }
            }
        }
    }

    printf("\nP\tBT\tWT\tTAT\n");
    for(int i = 0; i < n; i++)
        printf("P%d\t%d\t%d\t%d\n", i+1, bt[i], wt[i], tat[i]);

    return 0;
}
```